COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	9/18/2026	**DATE SUBMITTED:**	9/23/2026
NAME:	Larracas, Jerry Matthew L.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9 **BINARY SEARCH TREE**

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. I**ndicate your code screenshot and comments in table 9-1**

Code	Comment:

Task 2: Complete the following table:

Node	Height	Depth
A	3	0
B	2	1
C	2	1
D	2	1
E	2	1
F	2	1
G	2	1
H	1	2
I	1	2
J	1	2
K	1	2
L	1	2
M	1	2

N	1	2
P	0	3
Q	0	3

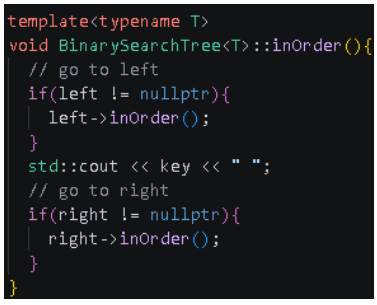
ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Task 3: After implementing the code for above, answer the following:

3.1 Given the tree diagram, find the result of the pre-order, post-order and in-order traversal strategies by hand. **Include this as table 9-3 in section**

Pre-order	A, G, C, B, D, F, E, J, I, H, P, L, K, N, M, Q
Post-order	A, B, C, D, E, F, G, H, I, J, K, L, M, N, P, Q
In-order	B, E, F, D, C, H, I, K, M, N, L, Q, P, J, G, A

3.2 Create a function for pre-order, post-order and in-order traversal. Make sure that each function displays an output into the console. Once you have the output, **create and fill table 9-4 in section 6** so that it contains the screenshot of the function, screenshot of the output and your observations. Your observations consist of a comparison between output in #1 and the output of your functions in #2

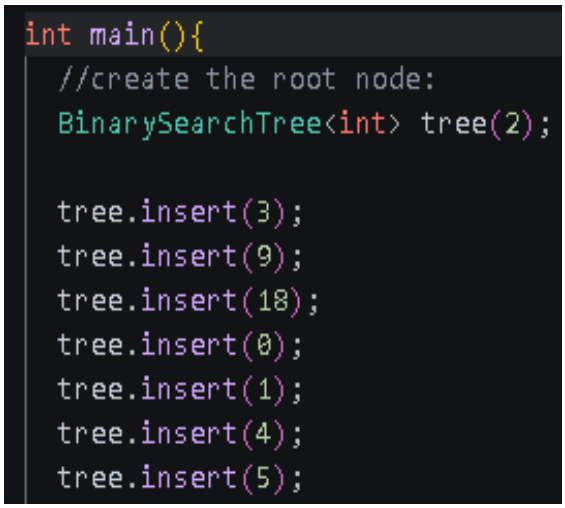
Functions	Outputs	Observations
		The function preOrder starts from the root and visits the left first then visits the right branch.
		The function inOrder simply searches the tree in order or sequentially.

		The function postOrder starts at the bottom of the tree until it reaches the root node.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.



```
int main(){
  //create the root node:
  BinarySearchTree<int> tree(2);

  tree.insert(3);
  tree.insert(9);
  tree.insert(18);
  tree.insert(0);
  tree.insert(1);
  tree.insert(4);
  tree.insert(5);
```

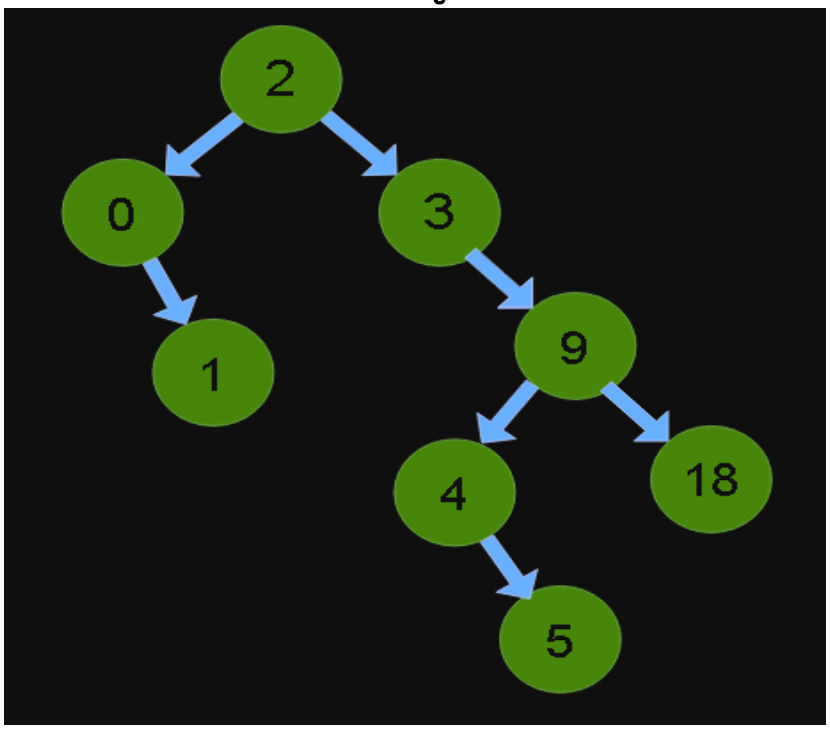


```
template<typename T>
void BinarySearchTree<T>::insert(T value){
    if(value < key){
        if(left == nullptr){
        left = new BinarySearchTree<T>(value);
      }else{
        left->insert(value);
      }
    }
    else if(value > key){
      if(right == nullptr){
        right = new BinarySearchTree<T>(value);
      }else{
        right->insert(value);
      }

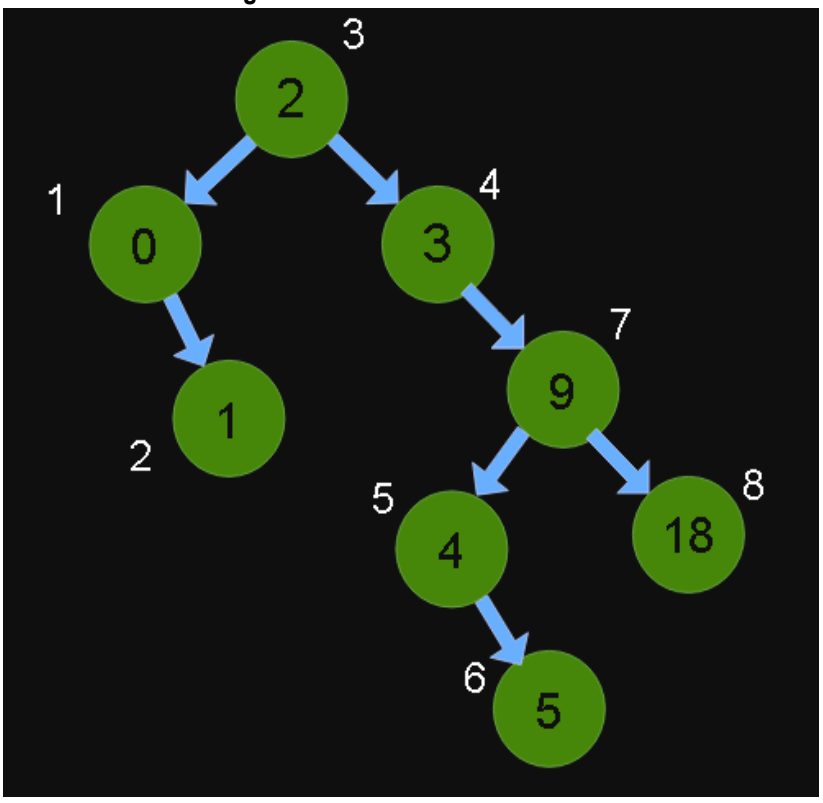
    }
}
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

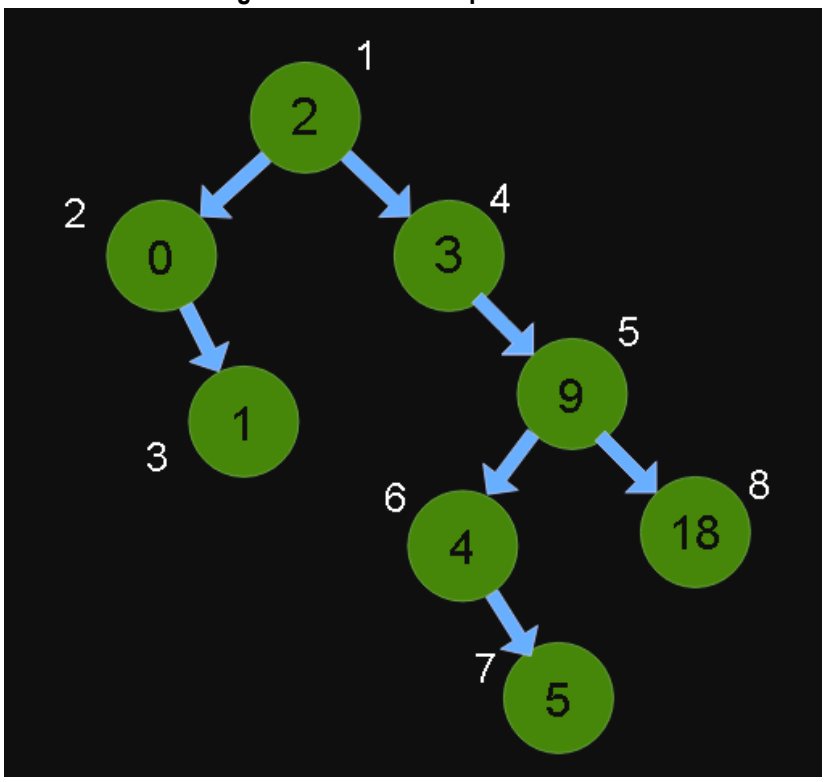
Tree Diagram



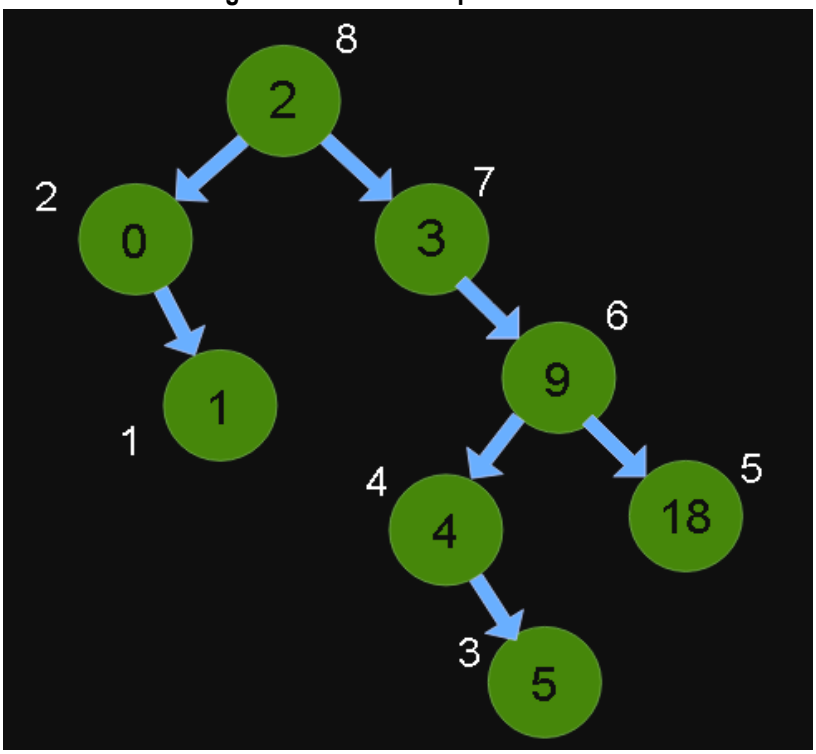
Tree Diagram with indicated in-order traversal



Tree Diagram with indicated pre-order traversal



Tree Diagram with indicated post-order traversal



Step 3: Compare the different traversal methods. In-order traversal was performed with what function? Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

In-Order Traversal

Created Function	**Console Output**

```
template<typename T>
void BinarySearchTree<T>::inOrder(){
  // go to left
  if(left != nullptr){
    left->inOrder();
  }
  std::cout << key << " ";
  // go to right
  if(right != nullptr){
    right->inOrder();
  }
}
```

```
inOrder: 0 1 2 3 4 5 9 18
```

Pre-Order Traversal

Created Function	**Console Output**

```
template<typename T>
void BinarySearchTree<T>::preOrder(){
  std::cout << key << " ";
  if(left != nullptr){
    left->preOrder();
  }
  if(right != nullptr){
    right->preOrder();
  }
}
```

```
preOrder: 2 0 1 3 9 4 5 18
```

Observation: I noticed that in pre-order traversal, its binary search starts at the root which is 2 then visits to the left first until it reaches to 1. After it's done checking its left, it will now visit to the right which is 3 to 9, then left again which is 4 to 5, then goes back to 9 and visits 18.

Post-Order Traversal

Created Function	**Console Output**

```
template<typename T>
void BinarySearchTree<T>::postOrder(){

  if(left != nullptr){
    left->postOrder();
  }

  if(right != nullptr){
    right->postOrder();
  }
  std::cout << key << " ";
}
```

```
postOrder: 1 0 5 4 18 9 3 2
```

Observation: In post-order traversal, the visiting procedure starts at the bottom on the left of the root which is 1 up to 1 and will not yet visit the root, it will instead visit the bottom node of the root's right which is 5 and all of the nodes that is on the right of the root until it visits it all, it will now visit the last value which is the root with the value of 2.

Conclusion:
I have learned in this activity to implement a binary search tree including with different traversals such as pre-order, in-order, and post-order. However, I wasn't able to complete the other procedure due to my lackness of implementation or failed to do the procedure on my own.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.